

# CSA1717-ARTIFICIAL INTELLIGENCE

## PRACTICAL LAB EXPERIMENTS

### RAGAVI S – (192565027)

#### **EXPERIMENT 9: TRAVELLING SALESMAN PROBLEM**

##### **Aim**

To write a python program to implement travelling salesman problem.

##### **Objective**

- To understand the concept of the Travelling Salesman Problem (TSP).
- To implement TSP using the brute-force approach in Python.
- To find the minimum-cost tour visiting all cities exactly once.

##### **Algorithm**

**Step 1:** Initialize the best distance and best route to be infinity and an empty list, respectively.

**Step 2:** Generate every possible permutation of the cities.

**Step 3:** For each permutation, calculate the distance of the path by summing the distances between each pair of consecutive cities in the permutation.

**Step 4:** Add the distance from the last city to the first city.

**Step 5:** If the calculated distance is shorter than the current best distance, update the best distance and best route.

**Step 6:** Return the best distance and best route.

##### **Source Code**

```
import itertools
```

```
def tour_cost(tour, graph):
```

```
    cost = 0
```

```
    for i in range(len(tour) - 1):
```

```
        cost += graph[tour[i]][tour[i + 1]]
```

```
cost += graph[tour[-1]][tour[0]]  
  
return cost
```

```
# Input
```

```
n = int(input("Enter number of cities: "))
```

```
graph = []
```

```
print("Enter the cost matrix:")
```

```
for i in range(n):
```

```
    row = list(map(int, input(f"Enter row {i}: ").split()))
```

```
    graph.append(row)
```

```
# Fix starting city as 0
```

```
cities = list(range(1, n))
```

```
min_cost = float('inf')
```

```
best_tour = []
```

```
# Generate all possible tours
```

```
for perm in itertools.permutations(cities):
```

```
    tour = [0] + list(perm)
```

```
    cost = tour_cost(tour, graph)
```

```
if cost < min_cost:
```

```
    min_cost = cost
```

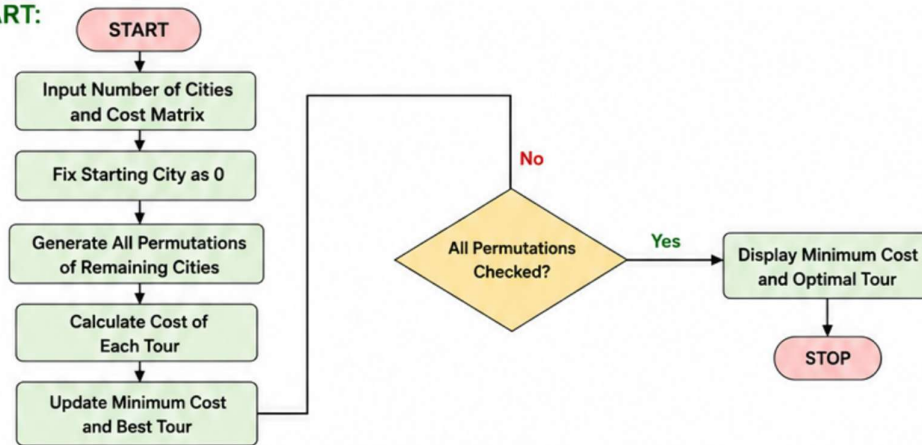
```
    best_tour = tour
```

```
print("\nMinimum Cost:", min_cost)
```

```
print("Optimal Tour:", " -> ".join(map(str, best_tour + [0])))
```

## Flowchart

FLOWCHART:



## Output

```
Enter number of cities: 4
Enter the cost matrix:
Enter row 0: 0 10 15 20
Enter row 1: 10 0 35 25
Enter row 2: 15 35 0 30
Enter row 3: 20 25 30 0

Minimum Cost: 80
Optimal Tour: 0 -> 1 -> 3 -> 2 -> 0
>>> |
```

## Result

The Travelling Salesman Problem was successfully implemented using the brute-force approach. The program calculated the minimum travel cost and displayed the optimal tour.